

Архитектурная логическая декомпозиция

Сценарий: создание бронирования авторизованным / неавторизованным пользователем
Проект: Lauberge GA

1. Назначение документа

Документ фиксирует логическую декомпозицию системы Lauberge GA для сценария создания бронирования гостем. Цель - определить бизнес-контексты, их ответственность, связи и рекомендации по реализации без преждевременного дробления системы на микросервисы.

Фокус: границы доменов, ownership данных, зависимости, риски доступности, оплаты, персональных данных и пригодность для дальнейшей детализации требований, API и архитектурных диаграмм.

2. Источники, вводные и ограничения

- Исходное описание задачи: создание бронирования авторизованным и неавторизованным пользователем.
- Проектное описание Lauberge GA: гостевые приложения Web/iOS/Android, приложение сотрудников, поиск, бронирование, оплата, номерной фонд, тарифы, услуги, роли, отчетность и НФТ.
- Рабочие вводные для MVP: система для сети отелей; guest booking разрешается политикой; основной сценарий - бронирование категории номера; hold на 10-15 минут; overbooking в гостевом канале запрещен.
- Ограничение: документ не проектирует API, БД, инфраструктуру, брокеры, кэши и конкретные технологии.

3. Подтвержденные вводные и рабочие допущения

ID	Вводная / допущение	Влияние
INP-01	Система проектируется для сети отелей, а не для одного объекта.	Все ключевые сущности должны учитывать hotel scope и добавление новых отелей.
INP-02	Гостевые каналы: Web, iOS, Android.	Бизнес-логика бронирования должна быть общей для всех клиентских приложений.
INP-03	Поддерживаются авторизованный пользователь и guest booking без аккаунта.	Нужно разделить Identity, Guest Profile и данные гостя внутри конкретной брони.
INP-04	Поиск и бронирование учитывают фактическую доступность номерного фонда.	Availability должен быть отдельным логическим контекстом.
INP-05	На оформление брони требуется временное удержание номера / категории.	Нужен Booking Hold как отдельная ответственность для снижения риска overbooking.
INP-06	Поддерживаются полная предоплата, частичная предоплата и оплата на месте.	Payment и Policies должны быть отделены от Booking, но согласованы по статусам.
ASM-01	В MVP Lauberge GA считается source of truth по броням, номерному фонду, тарифам и доступности.	Если появится внешняя PMS/CRS, границы Booking, Availability и Pricing нужно пересмотреть.
ASM-02	Основной публичный сценарий - бронирование категории номера; выбор конкретного номера опционален.	Availability и Hold должны поддерживать категорию как базовый уровень резервирования.
ASM-03	Snapshot цены, тарифа и политики фиксируется в момент создания брони.	Booking хранит зафиксированные условия, но не рассчитывает их самостоятельно.

4. Ключевые логические контексты

Контекст	Ответственность	Ключевые данные	MVP-реализация
Identity & Access	Аутентификация, роли, доступ сотрудников и гостей.	User, role, permission, hotel access scope.	Изолированный модуль или отдельный сервис при наличии корпоративного IAM.
Guest Profile	Профиль зарегистрированного гостя, контакты, предпочтения, связь с бронированиями.	Guest profile, contacts, preferences.	Модуль внутри backend. Не смешивать с guest data конкретной брони.
Hotel Catalog	Справочник отелей, публичные описания, адреса, фото, удобства.	Hotel, location, amenities, photos, rules summary.	Модуль внутри backend.
Search & Discovery	Поиск отелей и вариантов размещения по датам, гостям и фильтрам.	Search criteria, hotel cards, price from, availability summary.	Read-oriented модуль; при росте нагрузки кандидат на отдельный сервис.
Inventory	Физический номерной фонд: категории, комнаты, вместимость, атрибуты, статусы.	Room, room category, capacity, room status.	Модуль с четкой границей от Availability.
Availability	Расчет доступности по датам с учетом броней, hold, stop-sale и квот.	Availability calendar, quotas, stop-sale, available count.	Кандидат на отдельный сервис; в MVP - изолированный модуль.
Rates & Pricing	Тарифы, расчет стоимости проживания, налогов, сборов и обязательных услуг.	Rate plan, price breakdown, taxes, fees.	Модуль; при усложнении тарифов - отдельный сервис.
Policies	Правила guest booking, отмены, no-show, предоплаты и гарантий.	Cancellation policy, prepayment policy, no-show rules.	Модуль с централизованным применением правил.
Booking Hold	Временное удержание номера / категории на время оформления брони.	Hold ID, dates, category/room, expires at, status.	Изолированный модуль рядом с Availability/Booking.
Booking	Ядро бронирования: создание брони, номер бронирования, статусы, snapshot условий.	Booking, booking number, status, guest data, price/policy snapshot.	Ядро backend; в целевой архитектуре кандидат на отдельный сервис.
Payment	Платежный lifecycle: ожидание оплаты, оплата, ошибка, отмена, возврат.	Payment, status, amount, provider reference.	Изолированный модуль или отдельный сервис.
Notifications	Email/push/SMS уведомления по брони и оплате.	Notification, recipient, template, delivery status.	Модуль, не блокирующий создание брони.
Additional Services	Каталог и заказ дополнительных услуг к брони.	Service, service order, service price, service status.	Модуль; выделять отдельно при росте сервиса услуг.
Staff Operations	Канал сотрудника для операций с бронями, номерным фондом и платежами.	Staff action, operational task.	Orchestration layer, не отдельный source of truth.
Audit	Фиксация критичных действий гостей, сотрудников и администраторов.	Audit event, actor, action, object, timestamp.	Обязательный модуль для MVP.
Reporting & Analytics	Отчеты по загрузке, выручке, услугам, каналам и KPI.	Booking facts, revenue, occupancy, ADR, RevPAR.	Не включать в transactional path; в MVP - модуль/выгрузки.
Integration Layer	Адаптеры к платежному провайдеру, уведомлениям и будущим внешним системам.	External IDs, callbacks, sync status.	Только подтвержденные интеграции.

5. Главные границы ответственности

Правило	Архитектурное решение
Booking не рассчитывает цену	Booking получает итоговый price snapshot от Rates & Pricing и фиксирует его в составе брони.
Booking не хранит постоянный профиль гостя	Постоянные данные находятся в Guest Profile, а Booking хранит данные конкретного проживания.
Payment не является частью Booking	Booking инициирует оплату, но payment status и provider reference принадлежат Payment.
Notifications не блокируют создание брони	Сбой отправки подтверждения не откатывает корректно созданную бронь.

Правило	Архитектурное решение
Availability отделен от Inventory	Inventory знает, какие номера существуют; Availability знает, что доступно к продаже на даты.
Booking Hold отделен от Booking	Hold имеет отдельный короткий жизненный цикл и timeout.
Staff Operations не дублирует домены	Сотрудник работает через те же Booking, Payment, Inventory и Policies, что и гостевой канал.

6. Логические связи для сценария бронирования

Источник	Получатель	Что передается / зачем	Критичность
Guest App	Identity / Guest Profile	Определение режима: авторизованный пользователь или guest booking.	Высокая
Guest App	Search & Discovery	Параметры поиска: город/отель, даты, гости, фильтры.	Высокая
Search & Discovery	Availability + Pricing	Доступность и цена "от..." для карточек отелей.	Высокая
Guest App	Availability	Финальная проверка доступности выбранной категории/номера.	Критическая
Guest App	Rates & Pricing	Расчет итоговой стоимости перед созданием брони.	Критическая
Booking	Policies	Проверка guest booking, предоплаты, отмены и no-show.	Критическая
Guest App / Booking	Booking Hold	Создание временного удержания доступности.	Критическая
Booking	Booking Hold / Availability	Проверка и конвертация hold в бронь.	Критическая
Booking	Payment	Инициация оплаты, если тариф требует предоплату.	Высокая
Payment	Payment Provider Adapter	Передача платежа внешнему провайдеру и получение статуса.	Высокая
Booking / Payment	Notifications	Событие о создании брони или изменении оплаты.	Средняя
Booking / Payment / Staff	Audit	Фиксация критичных действий.	Высокая
Booking / Payment	Reporting	Факты для отчетности и аналитики.	Низкая для happy path

7. Сценарная проверка декомпозиции

Шаг	Контексты	Ответственность / риск
1. Пользователь входит или продолжает как гость	Identity, Guest Profile, Booking	Определить user mode; не смешивать guest booking с профилем.
2. Пользователь ищет отель	Search, Hotel Catalog, Availability, Pricing	Поиск должен быть быстрым и не выполнять transactional booking logic.
3. Пользователь выбирает даты, гостей и категорию	Inventory, Availability	Проверить вместимость и доступность.
4. Система рассчитывает стоимость	Rates & Pricing, Policies, Services	Перед созданием брони нужен финальный расчет, не только цена из поиска.
5. Пользователь вводит контактные данные и гостей	Booking, Guest Profile	Данные конкретной брони фиксируются отдельно от профиля.
6. Система проверяет политики	Policies, Booking, Payment	Проверить guest booking, предоплату, отмену, no-show.
7. Система ставит hold	Booking Hold, Availability	Защитить последний доступный номер/категорию от параллельной продажи.
8. Система создает бронь	Booking, Hold, Pricing, Policies	Сгенерировать номер брони, сохранить price/policy snapshot.
9. Система инициирует оплату	Payment, Provider Adapter	Создать платеж и обработать результат/ошибку.
10. Система отправляет подтверждение	Notifications, Booking	Уведомление не должно откатывать бронь при сбое доставки.
11. Бронь отображается в ЛК	Guest Profile, Booking	Для авторизованного пользователя бронь связана с аккаунтом.

8. Рекомендация по реализации в MVP

Для MVP рекомендуется модульный backend с четкими логическими границами. Не нужно сразу превращать каждый контекст в микросервис. Основная цель - не смешать правила бронирования, доступности, тарифов, платежей, профиля и уведомлений.

Группа	Контексты	Рекомендация
Ядро сценария	Booking, Availability, Booking Hold, Rates & Pricing, Policies, Payment	Проектировать как отдельные bounded contexts. В MVP могут быть модулями одного backend, но с изолированной бизнес-логикой.
Поддерживающие контексты	Identity, Guest Profile, Notifications, Audit, Reporting	Делать отдельными модулями. Notifications и Reporting не должны блокировать создание брони.
Административные каналы	Staff Operations, Admin Operations	Не делать источником истины. Они должны вызывать профильные домены.
Интеграции	Payment Provider Adapter, Notification Providers, будущие PMS/CRS/CRM	Изолировать через адаптеры. Не протаскивать внешние модели внутрь Booking.

9. Кандидаты на отдельные сервисы в целевой архитектуре

Контекст	Когда выделять отдельно	Почему
Booking	Когда появится несколько каналов создания/изменения брони и высокая критичность lifecycle.	Центральный домен с собственными статусами и правилами.
Availability	Когда поиск и проверка доступности начнут масштабироваться отдельно.	Критичный контекст для предотвращения overbooking.
Booking Hold	При высокой конкуренции за номерной фонд и большом количестве одновременных оформлений.	Короткий lifecycle, timeout и concurrency.
Payment	При нескольких провайдерах, сложных возвратах и строгих требованиях безопасности.	Отдельный жизненный цикл и интеграции с провайдерами.
Rates & Pricing	При усложнении тарифов, промо, валют, налогов и правил ценообразования.	Часто меняющиеся бизнес-правила.
Notifications	При нескольких каналах, шаблонах, retry и массовых уведомлениях.	Не должен нагружать Booking.

10. Архитектурные риски и рекомендации

ID	Риск	Контексты	Рекомендация
R-01	Overbooking из-за неконсистентной доступности.	Availability, Booking Hold, Booking	Создавать бронь только через актуальный hold или финальную атомарную проверку доступности.
R-02	Одновременное бронирование последнего номера.	Availability, Booking Hold	Hold должен уменьшать доступность на время оформления и освобождаться по timeout.
R-03	Расхождение цены между экраном и созданием брони.	Pricing, Booking	Перед созданием брони делать финальный расчет и сохранять price snapshot.
R-04	Бронь создана без зафиксированной политики отмены.	Policies, Booking	Сохранять policy snapshot в составе брони.
R-05	Сбой платежа после создания брони.	Booking, Payment	Разделить booking status и payment status; определить timeout неоплаченной брони.
R-06	Сбой уведомления после создания брони.	Notifications, Booking	Не откатывать бронь; хранить статус доставки и повторять отправку.
R-07	Несанкционированный доступ к ПДн гостя.	Identity, Guest Profile, Booking, Staff	Разграничить доступ по ролям и отелям; аудит критичных действий.
R-08	Смешение профиля гостя и данных конкретной брони.	Guest Profile, Booking	Профиль хранить отдельно; данные гостей в брони фиксировать как booking-specific.
R-09	Booking превращается в god-context.	Booking, Pricing, Payment, Notifications	Не размещать внутри Booking расчет цены, платежную обработку и отправку уведомлений.
R-10	Отчеты нагружают transactional backend.	Reporting, Booking, Payment	Reporting не должен участвовать в синхронном создании брони.

11. Итоговая архитектурная позиция

- Оптимальная стратегия для MVP - модульная архитектура с жесткими доменными границами, а не набор микросервисов ради микросервисов.
- Ядро сценария: Booking, Availability, Booking Hold, Rates & Pricing, Policies, Payment.
- Критично для надежности: Availability, Booking Hold, Booking, Payment.
- Критично для безопасности: Identity, Guest Profile, Booking, Payment, Staff Operations, Audit.
- Не дробить преждевременно: Hotel Catalog, Inventory, Guest Profile, Policies, Additional Services, Staff Operations, Reporting.
- Внешние интеграции изолировать через адаптеры, особенно платежного провайдера и провайдеров уведомлений.